

[< lewish](#)[light](#) [twitter](#) [github](#) [linkedin](#)

How to beat ARC-AGI-2

2025-07-14 / 23 min read

- > Understanding existing ARC approaches
 - > Framing the ARC problem
 - > The principal components of existing ARC approaches
 - > Program representations / program spaces
 - > Program search approaches
 - > Why TTT and Thinking models perform an implicit program search
- > What exactly is test time adaption?
 - > Understanding the test-time adaptation of O3 / Grok 4
- > The completeness vs search-tractability trade-off
- > Making smooth program spaces complete
- > Making search over discrete programs tractable
 - > Defining expressivity with respect to a set of programs
 - > Constructing rich program spaces through recombination
 - > Learning to move through rather than sample from the program space
- > Insights and hypotheses that will guide our work

Well unfortunately we don't know the answer, but this is our best attempt to draw out the most important decisions and to start laying out a few research directions that we (myself, and Ronan McGovern of [Trelis Research](#), and anyone who wishes to [join us](#)) are excited about working on. We hope by publishing this we can inspire others to work on ARC, get a better grasp of the fundamental challenges, and consider coming along on this journey with us!

Much of this post builds on top of my previous write up [recapping ARC and covering the existing research](#), and if you aren't familiar with ARC overall and the existing research, I would recommend starting there.

Understanding existing ARC approaches <#>

Framing the ARC problem <#>

For much of what follows, we describe ARC fundamentally as a program synthesis problem, even for transductive approaches using LLMs. To put a solution to ARC in its most fundamental form, you solve ARC by:

1. Defining a space of programs P
2. For each problem, find the shortest p_i within P such that for all example grid pairs, $p(\text{input}) = \text{output}$

The principal components of existing ARC approaches

The program synthesis framing leads us to the most important two features over which to compare and contrast a diverse set of approaches to ARC:

1. **Program representation:** How do you represent a program (and with that, the space of all programs), and what is your program executor/runtime, i.e. what process actually computes $p_i(\text{input})$ to produce the output? Some examples:
 - > Programs might be described in Python code, and executed with a Python interpreter.
 - > Programs might be implicitly described by the state+weights of a neural net (Transformer), and executed through successive token generation.
2. **Program search:** Given the space of programs defined by a program representation, how are you going to find the best program for the task at test-time? Some examples:
 - > Using an LLM to generate/sample python programs, conditioned on seeing the training examples for a given task.
 - > Heuristically driven search through python programs such as using a minimum-descriptor-length heuristic, pixel error, etc. on the output.

Below I will argue that LLMs implicitly model programs, just in a rather abstract program space of text, and that is why we see any form of test-time-fine-tuning (TTFT/TTT) or thinking models as a form of program search.

Under these two dimensions we can look at some of the real solutions to ARC from 2024, and see how each of them considers program representations and what their overall search procedure is.

Approach	Program representation	Program search
LLM + TTT (e.g	Neural: The "program" is	Gradient-Based (Explicit):

Approach**Program representation****Program search**

The [LLM Architect](#))

described by the model weights and a prompt, and is executed in a forward pass.

Adam/SGD optimizer performs a guided search on the model weights at test time by fine-tuning on the test examples (a process known as Test-Time Training or TTT).

Reasoning LLMs
(e.g 03)

Neural: The "program" is described by the model weights, prompt and thinking tokens, and is executed in a forward pass.

Learned (Implicit): The LLM's internal, pre-trained reasoning process acts as a form of guided search with backtracking.

Heuristic driven DSL search (e.g [Icecuber](#), [Object-MDL](#))

Discrete: A program constructed from a library in a custom, hand-crafted Domain Specific Language (DSL, a programming language specialized for a particular task), with an explicit runtime.

Hand-Crafted (Explicit): Heuristic driven search over the DSL, with hand-crafted pruning and prioritisation.

LLM-Guided program synthesis (e.g [Greenblatt](#))

Discrete: A program in a general-purpose language and runtime (Python, run with cpython).

Learned (Explicit): An LLM acts as a powerful, pre-trained sampler to generate and refine program candidates.

[Searching Latent Program Spaces](#)

Neural: A vector in a learned latent space. The program is executed by a learned decoder, producing an output grid.

Gradient-Based (Explicit): SGD performs local search over the latent program vector space at test time.

Neurally guided

Discrete: A program constructed from an adaptive

Learned (Explicit): A learnt "recognition model" guides

Approach	Program representation	Program search
Discrete program search (e.g. DreamCoder)	library in an initially hand-crafted DSL. Adaptive, as new library primitives are drawn from complete programs during training.	the search over the DSL.

Program representations / program spaces

The program representation falls primarily into two buckets - **discrete** or **continuous**. These two dimensions also effectively map onto inductive or transductive approaches respectively. **Inductive** approaches produce an interpretable, explanatory program, while **transductive** approaches generate the final output grid directly without an explicit intermediate program or "explanation".

Program representation has a significant impact on the subsequent search process as we'll cover below. There are many trade-offs, and as we saw with [one of the papers](#) last year, it is often the case that these two approaches are complementary.

Consideration	Discrete/Symbolic (e.g., Python, custom DSL)	Continuous/Neural (e.g., latent vectors, TTT)
Generalizability	Inherently high: Symbolic programs are defined by abstract rules. A <code>translate(x, y)</code> command generalizes perfectly to any object and grid size by definition.	Variable: Neural execution can be brittle and fail on inputs that deviate from the training data (e.g., larger grids), but when the training data is sufficiently broad, and an appropriate model architecture, are capable of generalizing well.
Completeness	Variable: Hand-coded ARC DSL program primitives (i.e. subprograms) have a risk of being incomplete, i.e. insufficient to solve all	Data dependent: Learnt program representations have a risk of being incomplete. The degree of completeness is determined

Consideration	Discrete/Symbolic (e.g., Python, custom DSL)	Continuous/Neural (e.g., latent vectors, TTT)
	conceivable ARC problems. Provably complete representations are easy to engineer, but come at the cost of a larger program search space.	by the training data, which may not sufficiently cover the test set. It is hard to prove that a continuous program space can completely cover all possible ARC test problems.
Precision	Perfect: A symbolic interpreter executes commands deterministically. There is no ambiguity or probabilistic error in the output.	Imperfect: Neural execution is probabilistic. The output can be "fuzzy", with small errors or artifacts, and may not be perfectly repeatable.
Compositionality	Natural: Discrete languages are inherently compositional. It is relatively easy to combine functions and primitives to build complex programs.	Difficult: While theoretically possible, learning compositional operators in a purely continuous space is an unsolved research problem. Composition may happen implicitly in some architectures.
Differentiability	Non-Differentiable: The process of executing a discrete program is a black box to gradient descent. SGD as a direct search mechanism is off the table.	Differentiable: The runtime that executes the program, e.g. a Transformer, can also be end-to-end differentiable, making it suitable for gradient based optimization.

We note that models using discrete combinations of neural primitives are possible in theory - e.g. neural primitives (continuous) with discrete structures on top (loops, conditions, etc) - but, to our knowledge, have not been explored.

Program search approaches

There is no doubt that the way to beat ARC is by searching intelligently/efficiently, particularly with ARC-AGI-2 explicitly attempting to close a gap of brute-force-ability.

What we saw in 2024 was a shift toward gradient-based search. We also saw the rise of thinking models that employ a learned search process. To summarize them briefly, before digging into some of the more subtle aspects of program search:

- > **Heuristic-driven discrete program search.** This is the original Icecuber approach, but also the foundation for [Object-centric Models and the MDL Principle](#), which guides its search procedure with a greedy minimum description length, or “greedy mdl”, objective. “greedy mdl” involves building program representations by tweaking one primitive at a time, keeping the representation that provides the shortest total length of that program plus its representation of the data. It is grounded in a belief that more compressive programs are likely to be correct ones.
- > **Gradient based optimization over continuous program spaces.** Gradient descent has gotten us this far, why not further? [Test-Time-Training \(TTT\)](#), and [Searching Latent Program Spaces \(SLPS\)](#) both use gradient descent to perform a search over the space of possible programs. This is quite powerful and, as of the time of writing, is leading the competition leaderboards. The missing piece here is likely completeness - can your model actually find and execute the programs needed for the test set?
- > **Learned search procedures.** Hand-coded heuristics or SGD are both fundamentally fixed search procedures - they haven't been learned from data. What if that search process itself can be learned? We see the process of a thinking model and its thinking tokens as performing a type of search, and as these models were specifically designed to “think” - they are leveraging that to propose, validate, refine and

back track on ideas. This is a form of meta-learning, currently quite expensive, but with the potential to be powerful.

Why TTT and Thinking models perform an implicit program

search

Both "thinking" language models and models adapted via Test-Time Tuning (TTT) can be understood as performing a form of implicit program search at inference time. While their goals are similar - to find a specific computational process that solves the task at hand - their mechanisms for doing so are quite different. They each explore a different kind of search space using a different kind of search algorithm, revealing a core trade-off in how models can adapt to new problems.

The key to recognizing this is that an LLM is literally - a program. It is a function that you can provide textual input and get back a textual response:

$$\text{output} = f(\text{prompt})$$

It's parameterized by two things - the model parameters (as well as its implicit architecture) and the input tokens:

$$\text{output} = f(\text{params}, \text{prompt})$$

There are two ways you can change this program - change the params, or change the input tokens, and as we see below, this maps to TTT approaches and thinking approaches respectively.

In the context of thinking models, you can think of splitting this into two programs, one for generating thinking tokens, and one that produces the final output (grid or program):

$$\text{thoughts} = g(\text{params}, \text{prompt})$$

$$\text{output} = f(\text{params}, \text{prompt}, \text{thoughts})$$

What exactly is test time adaption?

Approaches to ARC are often categorized by their ability to perform some form of test-time adaptation, but what exactly does that mean? What are we adapting to?

For any given problem you must of course adapt to the example data / train pairs, and this is necessary in any approach.

But, let's say you sample a solution (e.g a program) and it doesn't work - do you use that information to change how you draw your next sample? Asking an LLM to produce 10K sample python programs is not adaptive to this synthetic data, whereas asking an LLM to refine a program repeatedly based on its previous errors is.

So there is a baseline condition for test-time adaptivity, and that is that some form of search must happen at test time. But, what would it mean to be really test-time adaptive? I will go further and define a measure of test-time adaptivity as:

the degree to which the search process at any given step is conditioned on all previous steps

The prototype for test-time adaptation comes in the form of TTT/TTFT which delivered the top two private scores in ARC 2024. Within these approaches, the search space is the model parameter space, and SGD is the search method. By performing gradient descent repeatedly until convergence, your model produces attempts/samples in the forward pass, and then uses gradient descent in order to control the direction of the search. Where you end up in the loss landscape of parameters is dependent on your whole journey through it, and in that sense it has a fairly high degree of test time adaptivity.

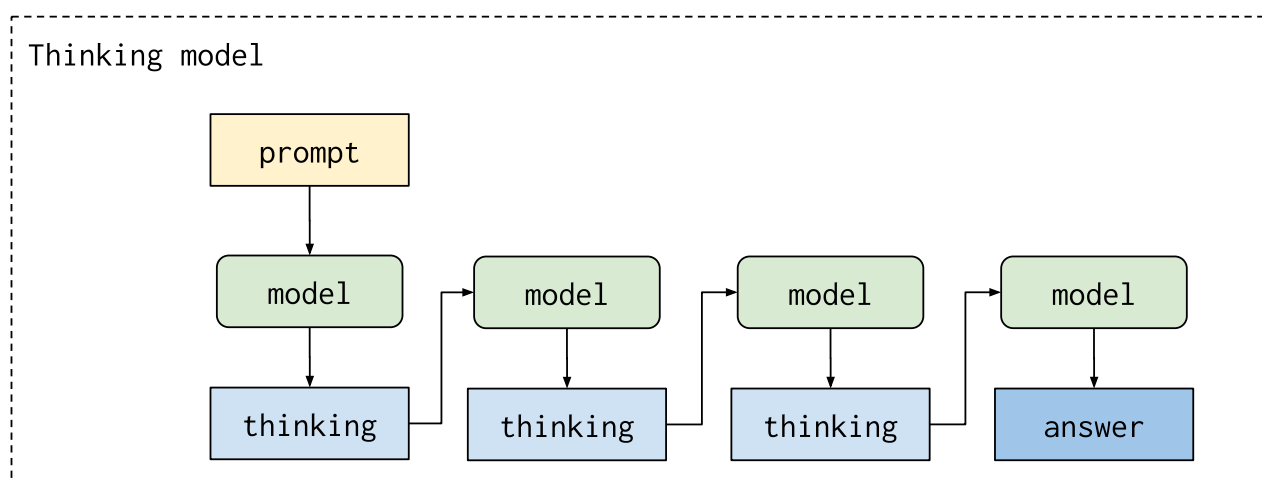
Understanding the test-time adaptation of O3 / Grok 4

There are two pretty hard to ignore data points on the ARC results leaderboards. Within the unrestricted compute category and semi-private test set:

- > In Dec 2024, A version of o3 fine-tuned for ARC on a very high compute setting [matched human performance on ARC-1](#).
- > In July 2025, Grok 4 launched with an announcement of [scoring 16% on ARC-2](#), matching or beating all specifically crafted approaches.

Reasoning models seem to be unreasonably effective. Why is that?

Our basic conceptual model of a thinking model is that it is performing a kind of search - as the model explores candidate solutions, self-validates, backtracks, samples new programs/answers - it is not only adjusting to the problem but learning from its own thinking and attempts, using a pre-trained model and in-context learning from tokens to guide the search, where the thinking tokens generated during search effectively "reparameterize" the model/program, rather than the weights themselves.



This is potentially a much more powerful search algorithm than gradient descent used in TTT:

1. In thinking models, the model has **explicitly learned how to search** rather than using a fixed optimizer (e.g. ADAM).
2. The **search happens over discrete tokens**, rather than through continuous representations, and this enables much more complex (and arguably turing-complete) computation to happen.
3. Each iteration in a thinking model is **conditioned on the entire history of search steps**, whereas SGD is generally happening based on local (or, a little broader than local, when using momentum) features.

If you want to take inspiration from these results, then a learned search process, and some element of discreteness - both seem like critical differentiating factors that need to be replicated.

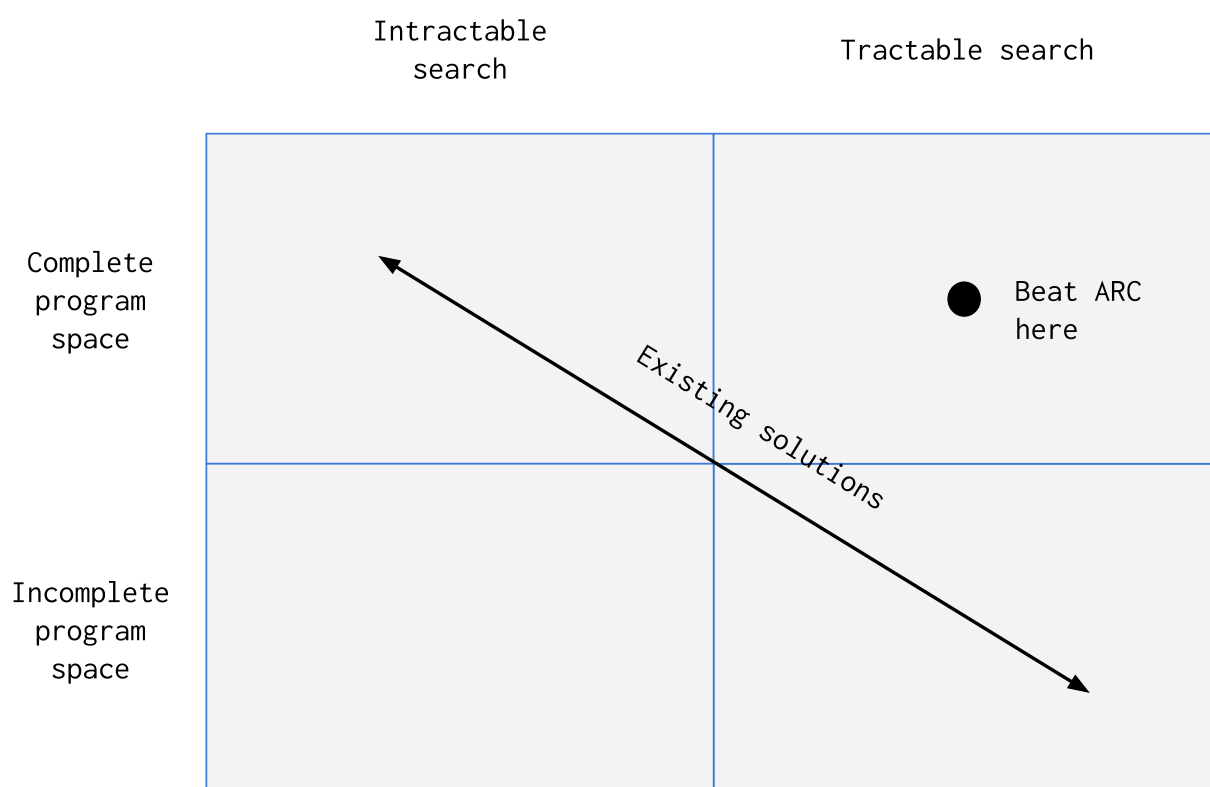
The completeness vs search-tractability trade-

off

From the above, what starts to emerge is a trade-off between two options:

1. Use a complete and discrete program representation, losing smoothness, and making search difficult
2. Limit the size of your search space, make it smooth, and search becomes tractable

How do you have your cake and eat it? How can we give each of these two different extremes what they are missing?



To achieve 100% on ARC, you need to search over a program space that captures the set of programs that are solutions to the test-time problems.

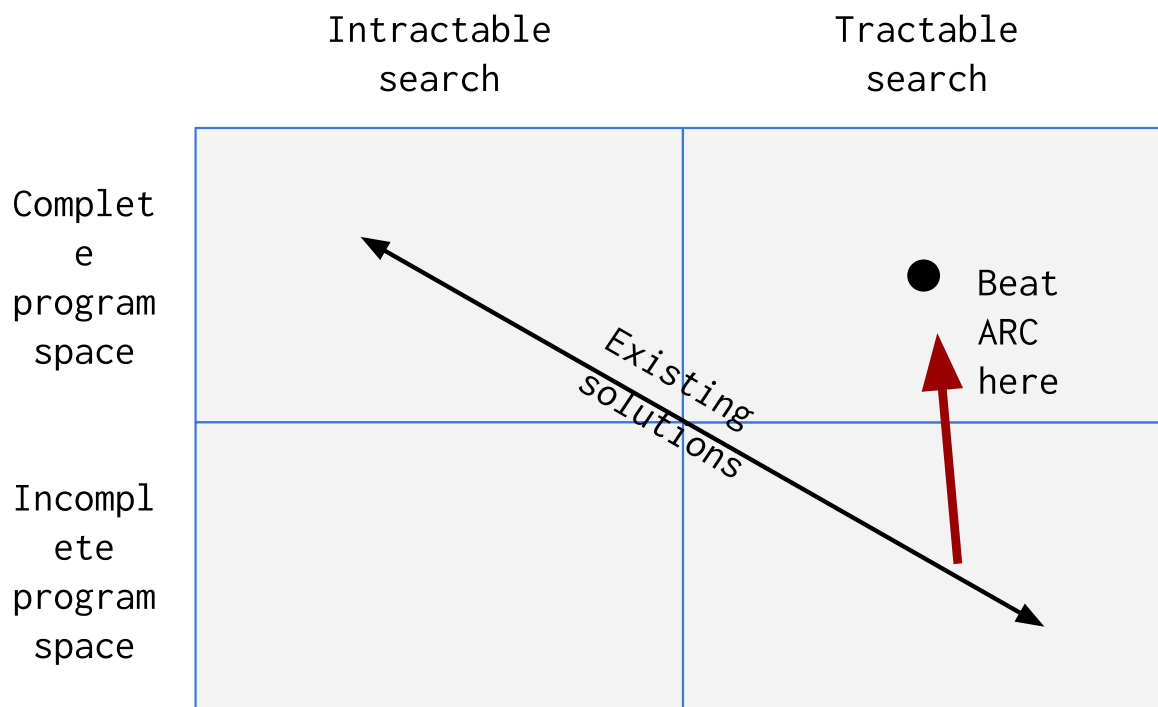
It is easy to make discrete programs complete. Turing completeness is not hard to achieve. If you construct a more tuned library of operations such as with [ARC-DSL](#), whilst you can be certain your set of primitives is sufficiently expressive to cover the training set, you can only hope it

also covers the test set. Ultimately though, **for discrete programs, completeness is not the primary challenge.**

Making differentiable programs complete is difficult. What can your model (e.g a neural net or a transformer) actually compute? With a fixed compute budget, [the scope of functions is limited](#). Any discrete operations (like `if` statements or `argmax`) aren't smooth, and once you produce discrete tokens (thinking models) you have a gradient problem. There are probably ways to mitigate this, to support more powerful yet smooth computation, in an attempt to give currently "tractable" search processes like TTT or SLPS the ability to generate a more complete set of programs.

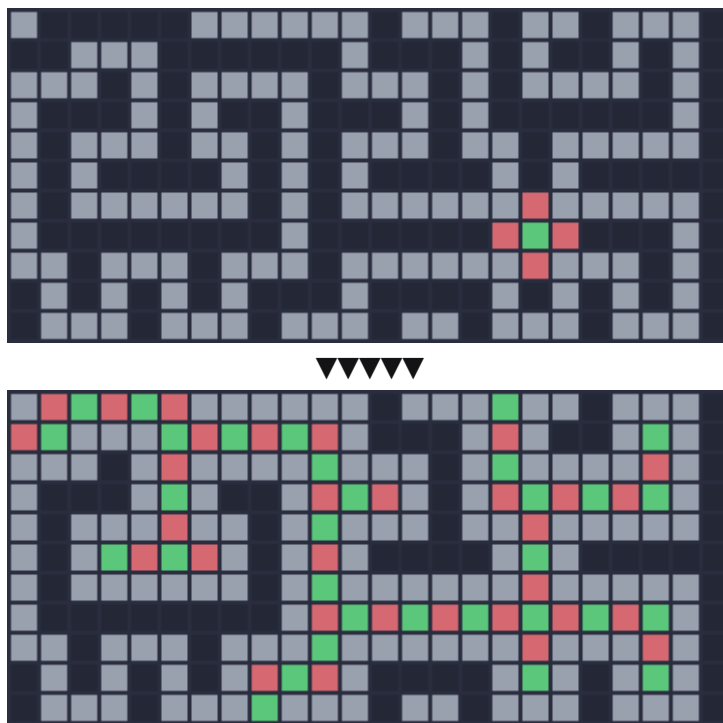
Making smooth program spaces complete

This is about building on top of smooth, gradient based approaches and increasing the scope of what they are actually capable of computing, for example transductive TTT, and Searching Latent Program Spaces, where the programs themselves are neural.



Rather than trying to make discrete program search tractable, how do we keep our beloved SGD, the GOAT search algorithm, and instead focus on making the space of programs it can discover more complete?

My canonical example for the kind of ARC problems where the limited compute of a single pass of a transformer can fall down is [b782dc8a](#):



How can a transformer trace out all of the paths of a maze in a single pass? Computation of a recursive form needs to be performed here, and there is no avoiding that.

One obvious, but probably naive and expensive approach to improving the computational power of a model is to simply stack them, with each invocation of the model performing some partial transformation (previously referred to as [refinement](#)). Provided that the partial representations are continuous (e.g using log probabilities rather than discretized grids) then the entire thing can be differentiable.

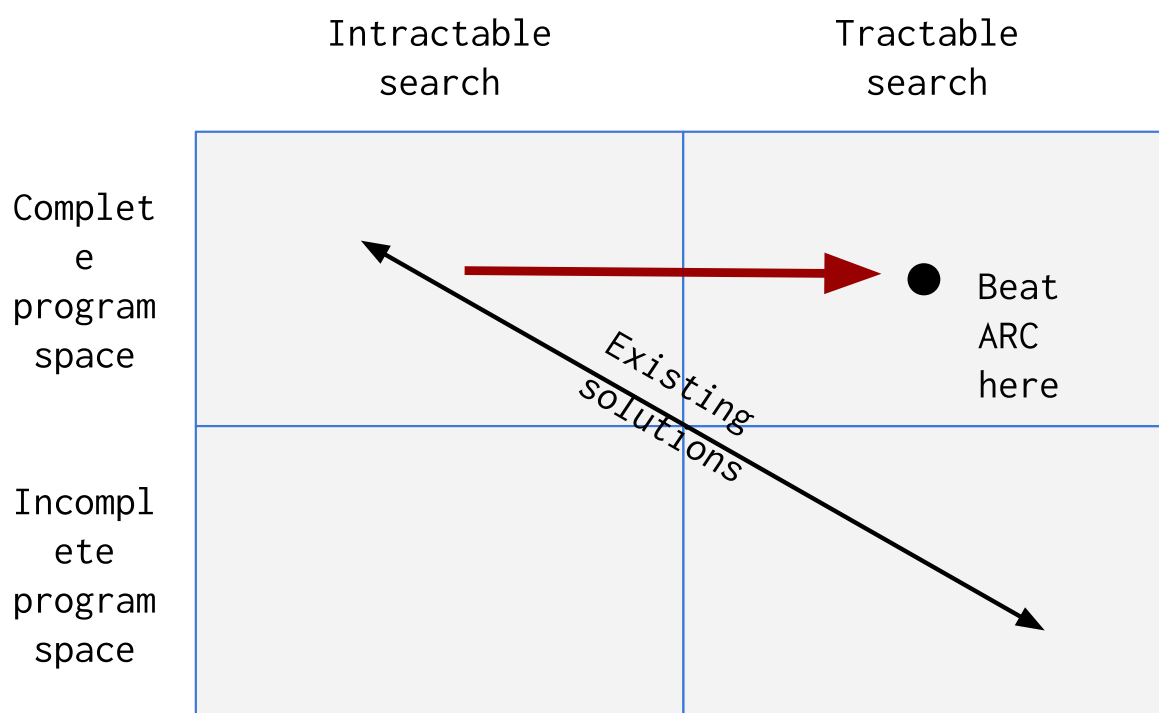
More complex, flexible, or efficient forms of differentiable computation are likely possible and this area is probably ripe for innovation. However, I have not explored this in depth as it's not where we plan to start and probably deserves its own write up.

Making search over discrete programs tractable

#

This is about how to more efficiently search over a large, likely complete, space of discrete programs. For example building on top of LLM driven

program synthesis, or other discrete methods like Icecuber.



There are likely several tricks required here, and I'll briefly dive into some of the considerations.

- > **Expressive** program representations that reduce the size of the effective search space, by ensuring a large set of programs can be represented in as few bits/tokens as possible.
- > Building up a large **library of program primitives** that are useful across ARC tasks, in order to improve the efficiency of search and the overall length of new programs.
- > Using a library of primitives to construct, search, or train over a much **larger program space through composition** of primitives, and evolution of existing programs.
- > A **learned, neurally guided search process**, a model that can suggest most relevant library primitives or entire programs, conditioned on both the example pairs and the search history.
- > Programs that cover grid **representations as well as transformations**, extracting, abstracting information in ARC grids in ways that make it useful to specific transformation tasks.
- > Other search based heuristics, for example leveraging the Minimum-Description-Length (MDL) principle to explore certain parts of the program space. This uses a compressive measure (i.e. length) of

candidate programs and residual errors (predicted minus ground truth output grids) to guide search.

Defining expressivity with respect to a set of programs

If you generated random Python code, it would most likely be invalid and not even compile. If you generated syntactically correct programs, the vast majority of them would either do nothing interesting, crash, or run forever.

In this sense, Python is not particularly compact, or "semantically dense", particularly in relation to solving ARC problems, and this is what we are calling expressiveness. It might be complete, but the length of the average program required to solve an ARC task would be a significant number of tokens, and this increases the search space substantially.

If you have a set of primitives p , and these can be composed together to form a set of full programs P , then you can define the **expressivity** of p with respect to P as:

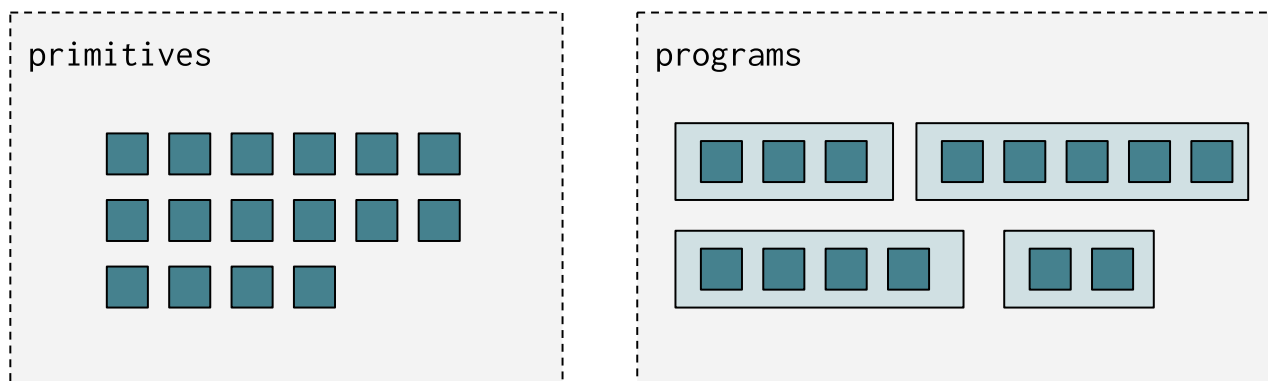
| The maximum number of primitives required to construct any program in

This effectively defines a kind of program depth, D , for some program space, and the size of your space is roughly proportional to $|p|^D$. All that to say - DSLs can be useful, as they can reduce the search space by reducing the number of primitives required to write a complete solution program for the given domain.

We note that there have not been any attempts we are aware of to augment LLM based program synthesis approaches with a DSL or library of primitives.

Constructing rich program spaces through recomposition

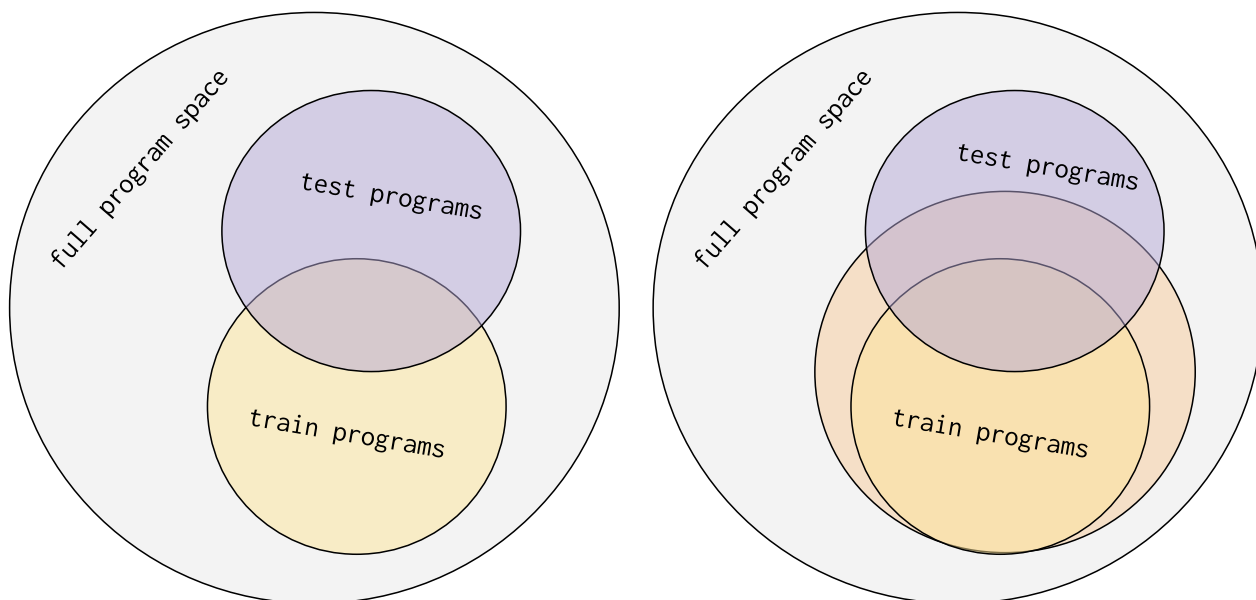
Through a set of primitives, we can define a set of programs as compositions of primitives:



We have such sets of primitives for ARC today, for example with ARC-DSL. Naively, you could attempt to train a model to predict complete programs based on the programs that you have that can solve the training set, but there isn't really any reason to expect that the required programs for the test set are an interpolation of the programs in your train set. In fact, it is our expectation that this is not the case and it's what ARC is specifically designed for.

By decomposing full programs into primitives, you can explore uncharted parts of the program space by recombining primitives to generate new programs (i.e textbook program synthesis). This can be done in two ways:

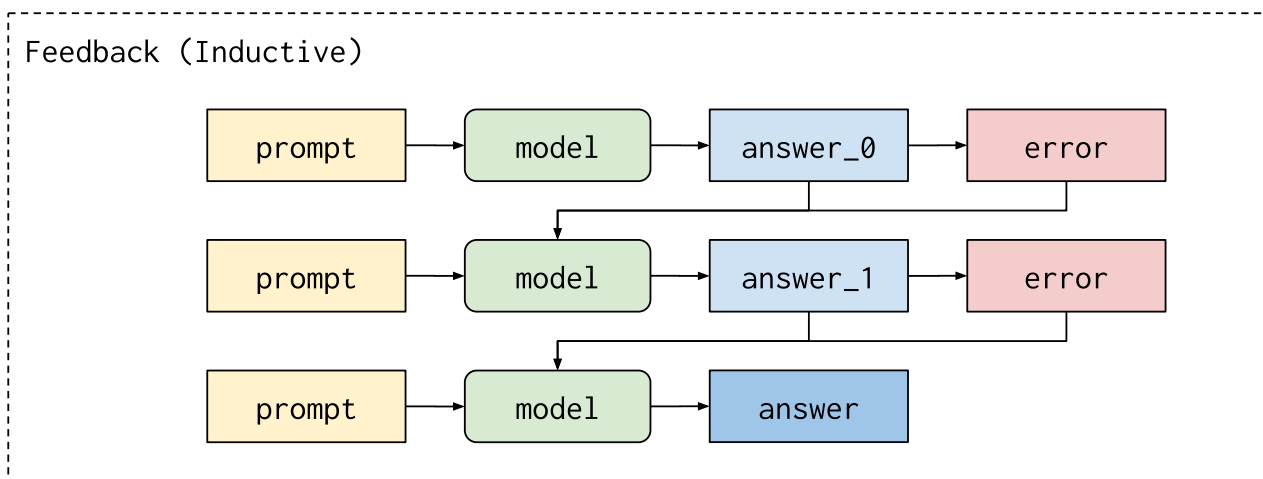
- > **Offline / during pre-training.** This allows you to take advantage of the unrestricted compute during pre-training, however - you have no feedback signal - you can explore the space randomly, and encode more of it into e.g the weights of an LLM, but you don't actually know if your expansion is covering the test programs. It's not smart, and it might even be considered cheating, but it can help.
- > **Online at test time.** As covered below, once you are in the test time environment you can in principle drive your search toward the test program space, leveraging some form of error signal to better guide the search - and this is where test-time adaptivity comes in, and the potential for much more efficient search processes.



You can make your space of programs covered during training bigger, but how do you make sure it moves towards the test program space?

Learning to move through rather than sample from the program space

Any model can be put in a feedback loop, and the hope here is that it should be easier (and more powerful) to learn how to move through the solution space rather than produce a one-shot answer from scratch - and this meets our criterion for good test-time adaptation. For an inductive approach, you can't obviously perform any kind of gradient based TTT, as you have no ground truth for the correct program, but you can do something like the following:



This is a simple way to give a model test-time adaptivity, and as discussed above, is roughly what we think is implicitly happening within thinking models.

Leveraging the recomposition of program primitives is one way to teach a model how to move through this space and build a synthetic dataset of program search trajectories. Alternatively, RL can be used to reinforce successful search paths, provided your model can actually find any solutions in the first place.

Insights and hypotheses that will guide our work

We are leaning toward approaching ARC as a neurally guided discrete inductive program synthesis problem - in line with Chollet's view, as [restated in his recent talk](#). These are strong opinions held weakly, and we are willing to change them, but it's a starting point on constraining our own meta-search.

1. We want to start with **inductive** approaches but leveraging deep learning as much as possible for guided search.
2. Investing in developing expressive and complete program **primitives** and methods to recompose them will be crucial to developing rich training datasets and efficient guided program synthesis.
3. We believe there are significant and unexplored opportunities in bringing test-time adaption to inductive approaches through **feedback** and learning program modification.

In terms of what this actually looks like, we expect to be focusing on some of the following over the coming months:

- > Building out or selecting a base set of primitives that is focused on expressivity and using this to generate large synthetic datasets of programs in line with methods like [ARC-Heavy \(BARC\)](#).
- > Exploring methods for program modification, testing against baseline LLMs to understand how well we can get them to respond to feedback and guide a search out of the box.
- > Fine-tuning our own models, likely LLMs to start with, to understand how well they can be adapted to use a library of primitives and the impact this can have on performance.
- > Fine-tuning models again within the feedback regime to see how efficiently we can learn to search, through RL or synthetic search trajectory datasets.
- > Exploring different or custom model architectures for both feedback and sampling, with a focus on better approaches to representing or seeing grids (e.g vision) as well as overall compute efficiency.

If you are interested in working on ARC, we'd love to talk!

[reply via email](#) [twitter](#)

[< lewish](#)